Assignment: Database-Integrated REST API

Introduction

In this assignment, we will build a **REST API** using **Express.js**, **Prisma** (ORM), **PostgreSQL**, and **Redis** for caching. The app integrates **Zod** for validation and includes comprehensive error handling, ensuring robustness and maintainability. By the end, you'll have a functional backend service that demonstrates CRUD operations, database integration, caching mechanisms, and modular project structure in a real-world scenario.

Working with Dev Container

To complete this assignment in a reliable and fully configured environment, please refer to the instructions in the file: `README-devcontainer.md`. This guide walks you through opening the assignment in **Visual Studio Code** using a **Dev Container**, which automatically installs all necessary Python libraries and tools. Following that setup ensures that the notebook runs smoothly without manual configuration or missing dependencies. Make sure to open the **main assignment folder** in VS Code and follow the steps outlined in the Dev Container guide before starting the notebook.

Starter Files

The initial code is available inside the `start` folder under the `code` folder associated with this activity. However, it's recommended to start the project from scratch.

Requirements

We'll be working with Express.js, Prisma, PostgreSQL, and Redis to develop our Product Inventory API. Here's what we need to accomplish:

Set Up the Development Environment

We need to:

- Initialize a Node.js project using `npm` with TypeScript support.
- Install necessary dependencies, including Express.js for routing, Prisma for ORM, Redis for caching, and Zod for validation.
- Configure scripts for development and production environments.

Build the Core Features

We need to implement the following functionalities:

1. **Database Integration:**
    - Use Prisma to define a `Product` model and connect it to a PostgreSQL database.
    - Handle database migrations and schema updates with Prisma.
2. **CRUD Operations:**
    - Implement endpoints for creating, reading, updating, and deleting products.
    - Use Prisma's type-safe queries for database interactions.
3. **Caching with Redis:**
    - Cache responses for read-heavy endpoints to optimize performance.
    - Invalidate cache when data changes (e.g., after create, update, or delete operations).
4. **Modular Project Structure:**
    - Organize the project into controllers, routes, utils, and validations.
    - Use TypeScript to enforce type checks across the application.
5. **Validation and Error Handling:**
    - Validate incoming requests using Zod schemas.
    - Handle errors gracefully with centralized error handling middleware.
6. **Middleware and Logging:**
    - Use `cors` for cross-origin resource sharing.
    - Use `morgan` for logging HTTP requests.

Test the Application

We need to verify:

- The API correctly performs CRUD operations on the `Product` resource.
- Redis caching optimizes response times for read-heavy endpoints.
- Validation ensures only valid data is processed.
- Errors are handled gracefully with meaningful responses.
- All tests pass successfully using tools like Postman or cURL.

Deliverables

The deliverable of this assignment is a working backend service that meets all the requirements above. We need to submit:

- The public GitHub repository containing the source code.
- Screenshots showing:
    - The API running locally.
    - Responses from API endpoints.
    - Redis caching in action.
- A brief README file explaining how to set up and run the app locally.
- Simple documentation for the app's functionality and testing process.

Conclusion

Building a **Product Inventory API with Caching** using Express.js, Prisma, PostgreSQL, and Redis is an excellent way to practice creating scalable backend services. By completing this activity, you've learned how to integrate a database, implement CRUD operations, optimize performance with caching, and structure a modular backend project. These skills form the foundation for developing more complex and efficient APIs in the future.